REPORTE: ENCADENAMIENTO EN LA BIBLIOTECA

1.Por que Becario(Ana) sale en el encadenamiento hacia adelante de AccesoNoche(Ana) y no en el de hacia atras?

Porque hacia adelante trabaja con todo lo que tiene enfrente: ve que Ana es estudiante y tiene credencial, asi que dispara la regla de beca aunque nadie se lo haya pedido. En cambio, hacia atras solo le importa resolver la meta (AccesoNoche), asi que busca solo las reglas necesarias para el acceso y pasa completamente de largo de la regla de beca.

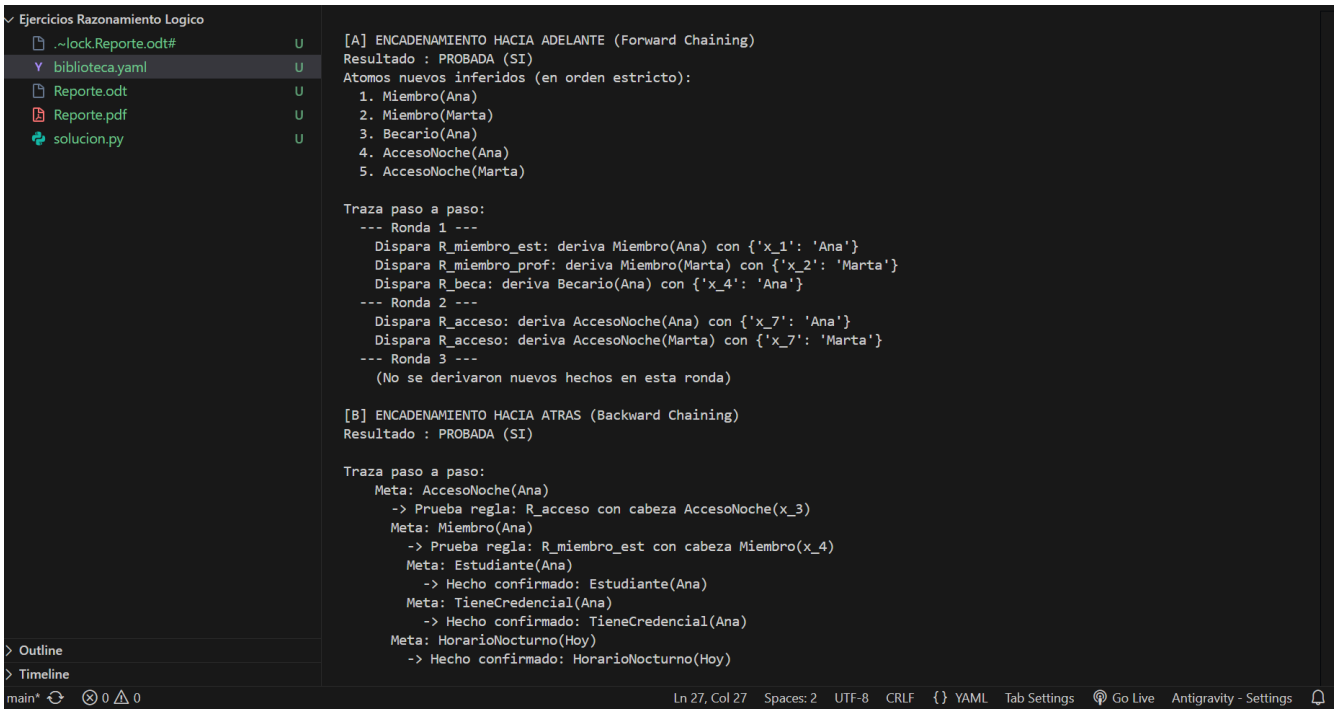
2.Por que hacia atras prueba R_miembro_est al buscar Miembro(Marta), si Marta es profesora?

Porque el algoritmo no sabe de antemano que Marta es profe; simplemente sigue el orden de la lista de reglas de arriba a abajo. Como la primera regla que habla de ser "Miembro" es la de estudiantes, prueba esa primero. Cuando se da cuenta de que Marta no es estudiante, descarta esa opcion y recien ahi pasa a la siguiente regla, que es la de profesor, donde si tiene exito.

3.Si se anade el hecho TieneCredencial(Luis) y se vuelve a preguntar AccesoNoche(Luis), que atomos nuevos aparecen y la consulta queda probada?

Si agregamos TieneCredencial(Luis), la consulta AccesoNoche(Luis) ahora SI queda probada. Al tener credencial, el sistema ahora puede deducir que Luis es miembro, que tambien tiene beca y que finalmente puede pasar de noche. Los tres atomos nuevos que se derivan son: Miembro(Luis), Becario(Luis) y AccesoNoche(Luis lara je).

Trazas :



```
CONSULTA: AccesoNoche(Luis)
================================================================

[A] ENCADENAMIENTO HACIA ADELANTE (Forward Chaining)
Resultado : NO PROBADA (NO)
Atomos nuevos inferidos (en orden estricto):
  1. Miembro(Ana)
  2. Miembro(Marta)
  3. Becario(Ana)
  4. AccesoNoche(Ana)
  5. AccesoNoche(Marta)

Traza paso a paso:
  --- Ronda 1 ---
    Dispara R_miembro_est: deriva Miembro(Ana) con {'x_1': 'Ana'}
    Dispara R_miembro_prof: deriva Miembro(Marta) con {'x_2': 'Marta'}
    Dispara R_beca: deriva Becario(Ana) con {'x_4': 'Ana'}
  --- Ronda 2 ---
    Dispara R_acceso: deriva AccesoNoche(Ana) con {'x_7': 'Ana'}
    Dispara R_acceso: deriva AccesoNoche(Marta) con {'x_7': 'Marta'}
  --- Ronda 3 ---
    (No se derivaron nuevos hechos en esta ronda)

[B] ENCADENAMIENTO HACIA ATRAS (Backward Chaining)
Resultado : NO PROBADA (NO)

Traza paso a paso:
    Meta: AccesoNoche(Luis)
      -> Prueba regla: R_acceso con cabeza AccesoNoche(x_3)
      Meta: Miembro(Luis)
        -> Prueba regla: R_miembro_est con cabeza Miembro(x_4)
        Meta: Estudiante(Luis)
          -> Hecho confirmado: Estudiante(Luis)
        Meta: TieneCredencial(Luis)
          [FALLO] No se pudo probar: TieneCredencial(Luis)
        -> Prueba regla: R_miembro_prof con cabeza Miembro(x_9)
```

```
      Meta: Profesor(Luis)
        [FALLO] No se pudo probar: Profesor(Luis)
      [FALLO] No se pudo probar: Miembro(Luis)
    [FALLO] No se pudo probar: AccesoNoche(Luis)
```

```
CONSULTA: AccesoNoche(Marta)
============================================================

[A] ENCADENAMIENTO HACIA ADELANTE (Forward Chaining)
Resultado : PROBADA (SI)
Atomos nuevos inferidos (en orden estricto):
  1. Miembro(Ana)
  2. Miembro(Marta)
  3. Becario(Ana)
  4. AccesoNoche(Ana)
  5. AccesoNoche(Marta)

Traza paso a paso:
  --- Ronda 1 ---
    Dispara R_miembro_est: deriva Miembro(Ana) con {'x_1': 'Ana'}
    Dispara R_miembro_prof: deriva Miembro(Marta) con {'x_2': 'Marta'}
    Dispara R_beca: deriva Becario(Ana) con {'x_4': 'Ana'}
  --- Ronda 2 ---
    Dispara R_acceso: deriva AccesoNoche(Ana) con {'x_7': 'Ana'}
    Dispara R_acceso: deriva AccesoNoche(Marta) con {'x_7': 'Marta'}
  --- Ronda 3 ---
    (No se derivaron nuevos hechos en esta ronda)

[B] ENCADENAMIENTO HACIA ATRAS (Backward Chaining)
Resultado : PROBADA (SI)

Traza paso a paso:
    Meta: AccesoNoche(Marta)
      -> Prueba regla: R_acceso con cabeza AccesoNoche(x_3)
      Meta: Miembro(Marta)
        -> Prueba regla: R_miembro_est con cabeza Miembro(x_4)
        Meta: Estudiante(Marta)

[B] ENCADENAMIENTO HACIA ATRAS (Backward Chaining)
Resultado : PROBADA (SI)
```

```
Traza paso a paso:
    Meta: AccesoNoche(Marta)
      -> Prueba regla: R_acceso con cabeza AccesoNoche(x_3)
      Meta: Miembro(Marta)
        -> Prueba regla: R_miembro_est con cabeza Miembro(x_4)
        Meta: Estudiante(Marta)
[B] ENCADENAMIENTO HACIA ATRAS (Backward Chaining)
Resultado : PROBADA (SI)

Traza paso a paso:
    Meta: AccesoNoche(Marta)
      -> Prueba regla: R_acceso con cabeza AccesoNoche(x_3)
      Meta: Miembro(Marta)
        -> Prueba regla: R_miembro_est con cabeza Miembro(x_4)
        Meta: Estudiante(Marta)

Traza paso a paso:
    Meta: AccesoNoche(Marta)
      -> Prueba regla: R_acceso con cabeza AccesoNoche(x_3)
      Meta: Miembro(Marta)
        -> Prueba regla: R_miembro_est con cabeza Miembro(x_4)
        Meta: Estudiante(Marta)
      -> Prueba regla: R_acceso con cabeza AccesoNoche(x_3)
      Meta: Miembro(Marta)
        -> Prueba regla: R_miembro_est con cabeza Miembro(x_4)
        Meta: Estudiante(Marta)
          [FALLO] No se pudo probar: Estudiante(Marta)
        -> Prueba regla: R_miembro_prof con cabeza Miembro(x_9)
      Meta: Miembro(Marta)
        -> Prueba regla: R_miembro_est con cabeza Miembro(x_4)
        Meta: Estudiante(Marta)
          [FALLO] No se pudo probar: Estudiante(Marta)
        -> Prueba regla: R_miembro_prof con cabeza Miembro(x_9)
        -> Prueba regla: R_miembro_est con cabeza Miembro(x_4)
        Meta: Estudiante(Marta)
          [FALLO] No se pudo probar: Estudiante(Marta)
```

```
  -> Prueba regla: R_miembro_prof con cabeza Miembro(x_9)
    [FALLO] No se pudo probar: Estudiante(Marta)
  -> Prueba regla: R_miembro_prof con cabeza Miembro(x_9)
  Meta: Profesor(Marta)
  Meta: Profesor(Marta)
    -> Hecho confirmado: Profesor(Marta)
    -> Hecho confirmado: Profesor(Marta)
Meta: HorarioNocturno(Hoy)
    -> Hecho confirmado: Profesor(Marta)
    -> Hecho confirmado: Profesor(Marta)
    -> Hecho confirmado: Profesor(Marta)
    -> Hecho confirmado: Profesor(Marta)
    -> Hecho confirmado: Profesor(Marta)
    -> Hecho confirmado: Profesor(Marta)
Meta: HorarioNocturno(Hoy)
  -> Hecho confirmado: HorarioNocturno(Hoy)
```